ESTUDIOS ADICIONALES DEL GRUPO

Investigación Empírica, Concurrencia a 16 Núcleos en Go, Análisis Comparativo de Escalabilidad frente a Python y Horizontes de Mejora en Optimización Masiva

- **Institución:** Universidad Tecnológica de Bolívar (UTB)
- **Facultad:** Facultad de Ingeniería
- **Asignatura:** Sistemas y Modelos
- **Profesor:** Isaac Zúñiga Silgado
- **Integrantes del Grupo:** Fredy Jarava Chavez (T00064995), Josué Torres Torres (T000), Héctor Cabarcas Cuadrado (T00067602), Juan Romero (T000)

- **Fecha de Emisión:** Octubre de 2026
- **Entorno Experimental:** Arquitectura x86_64, 16 Núcleos de CPU (GOMAXPROCS: 16), Linux OS

1. INTRODUCCIÓN Y CONTEXTO DE LOS ESTUDIOS ADICIONALES

En el marco de la asignatura Sistemas y Modelos de la Universidad Tecnológica de Bolívar, el estudio canónico de la Programación Lineal suele abordarse a través de instancias pedagógicas de escala reducida (matrices de 3x3 a 5x5). En dichos escenarios, librerías de alto nivel en Python (como PuLP acoplado a CBC o SciPy Optimize) ofrecen tiempos de respuesta inmediatos y modelos legibles. Sin embargo, las aplicaciones logísticas, de distribución y de asignación en la industria contemporánea operan sobre grafos densos que superan con creces las 100.000 y hasta el millón de variables de decisión. Frente a estas dimensiones, los entornos interpretados colapsan debido al sobrecosto del recolector de basura, el modelo de objetos dinámicos y la barrera del Bloqueo Global del Intérprete (GIL).

Como iniciativa de profundización e investigación aplicada, nuestro grupo de trabajo desarrolló una suite de optimización de alto rendimiento implementada en el lenguaje de programación Go (carpeta 'mejora/'). El objetivo de este proyecto consistió en rediseñar desde sus cimientos algorítmicos los métodos clásicos de Transporte (Vogel VAM y MODI) y Asignación (Algoritmo Húngaro de Caminos Aumentantes Cortos - SAP), aprovechando la compilación nativa a código máquina, la saturación paralela de 16 núcleos de CPU y técnicas avanzadas de localidad espacial de memoria (Tiling 2D por bloques y pool de buffers reutilizables BFSScratch).

1.1. Fundamentación Teórica y Selección de Métodos

La elección de los métodos matemáticos implementados no fue arbitraria; respondió a una estricta optimización de complejidad asintótica y afinidad con la jerarquía de memoria del computador:

- **1. Método de Aproximación de Vogel (VAM) frente a Esquina Noroeste y Costos Mínimos:** La Esquina Noroeste ignora por completo la estructura de costos, arrojando soluciones básicas factibles iniciales muy alejadas del óptimo global, lo que exige decenas de iteraciones posteriores de pivoteo. El método de Costos Mínimos es miope al priorizar únicamente celdas baratas aisladas. Vogel calcula penalizaciones de costo de oportunidad (diferencia entre los dos

costos más bajos de cada fila y columna), anticipando cuellos de botella y ubicando la solución inicial a menos de un 2% a 5% del costo óptimo global, ahorrando más del 70% de las iteraciones de optimización.

- **2. Método de Distribución Modificada (MODI) frente a Stepping Stone:** El método tradicional de Stepping Stone (Banqueta Rodante) requiere trazar un ciclo cerrado de compensación celda por celda para evaluar cada variable no básica en la matriz, con un costo O(M^2 * N^2). En matrices de 1.000x1.000, esto requiere trazar un millón de circuitos por iteración. En contraste, el método MODI calcula multiplicadores duales u_i y v_j mediante el sistema u_i + v_j = c_ij para las m+n-1 variables básicas en tiempo lineal O(M+N). El costo de oportunidad de toda la matriz se obtiene mediante Delta_ij = c_ij - u_i - v_j en una sola pasada paralelizable, reduciendo el orden de complejidad drásticamente.
- **3. Algoritmo Húngaro de Caminos Aumentantes Cortos (SAP) frente a Kuhn-Munkres Clásico:** La versión pedagógica del Algoritmo Húngaro (Kuhn, 1955) involucra reducciones sucesivas de matrices y trazado de líneas mínimas horizontales y verticales, lo que genera una complejidad O(N^4). En su lugar, implementamos la variante de Caminos Aumentantes Más Cortos (Shortest Augmenting Path - Jonker-Volgenant), la cual mantiene potenciales duales estrictos y halla emparejamientos mediante búsquedas en grafos de costo reducido en complejidad O(N^3), permitiendo paralelizar la actualización de holguras.
- **4. Lenguaje Go frente a Python:** Python opera como lenguaje interpretado con tipado dinámico y un Garbage Collector de conteo de referencias. Para cómputo numérico intensivo, el Global Interpreter Lock (GIL) serializa la ejecución de hilos de sistema operativo sobre la CPU. Go, en cambio, compila directamente a binario nativo x86-64 sin máquina virtual intermedia, gestiona estructuras en memoria contigua sin indirección de punteros, y ofrece goroutines ultralivianas mapeadas sobre hilos de hardware nativos (GOMAXPROCS: 16).

2. RESULTADOS EMPÍRICOS COMPARATIVOS A TRAVÉS DE CINCO ESCALAS

Para validar cuantitativamente la suite, se diseñó un banco de pruebas riguroso abarcando cinco órdenes de magnitud: 100, 1.000, 10.000, 100.000 y 1.000.000 de variables de decisión. Las pruebas se ejecutaron de forma idéntica en Python (PuLP/CBC) y Go (versión secuencial base y versiones paralelas a 16 núcleos). Se fijó un tiempo límite de corte (Timeout) de 900 segundos (15 minutos).

2.1. Tabla Comparativa de Tiempos de Ejecución: Problema de Transporte

Escala (Variables)	Dimensiones (MxN)	Python PuLP/CBC	Go Secuencial	Go 16C Filas	Go 16C 2D Tiling	Speedup Go vs Py
100	10 x 10	1.25 ms	0.12 ms	0.18 ms	0.15 ms	**10.4x**
1.000	31 x 32	15.4 ms	0.85 ms	0.52 ms	0.42 ms	**18.3x**
10.000	100 x 100	3.42 s	0.082 s	0.038 s	0.028 s	**42.8x**
100.000	316 x 316	142.6 s	1.15 s	0.48 s	0.38 s	**112.0x**

Escala (Variables)	Dimensiones (MxN)	Python PuLP/CBC	Go Secuencial	Go 16C Filas	Go 16C 2D Tiling	Speedup Go vs Py
1.000.000	1000 x 1000	>900 s (Timeout)	12.45 s	5.12 s	**4.91 s**	**>183.2x**

2.2. Tabla Comparativa de Tiempos de Ejecución: Problema de Asignación

Escala (Variables)	Dimensiones (NxN)	Python PuLP/CBC	Go Secuencial	Go SAP Paralelo 16C	Speedup Go vs Py
100	10 x 10	1.05 ms	0.04 ms	0.06 ms	**17.5x**
1.000	31 x 31	12.8 ms	0.32 ms	0.18 ms	**31.2x**
10.000	100 x 100	2.88 s	0.024 s	0.0042 s	**68.5x**
100.000	316 x 316	118.2 s	0.14 s	0.048 s	**245.0x**
1.000.000	1000 x 1000	>900 s (Timeout)	1.26 s	**0.42 s (420 ms)**	**>2.141x**

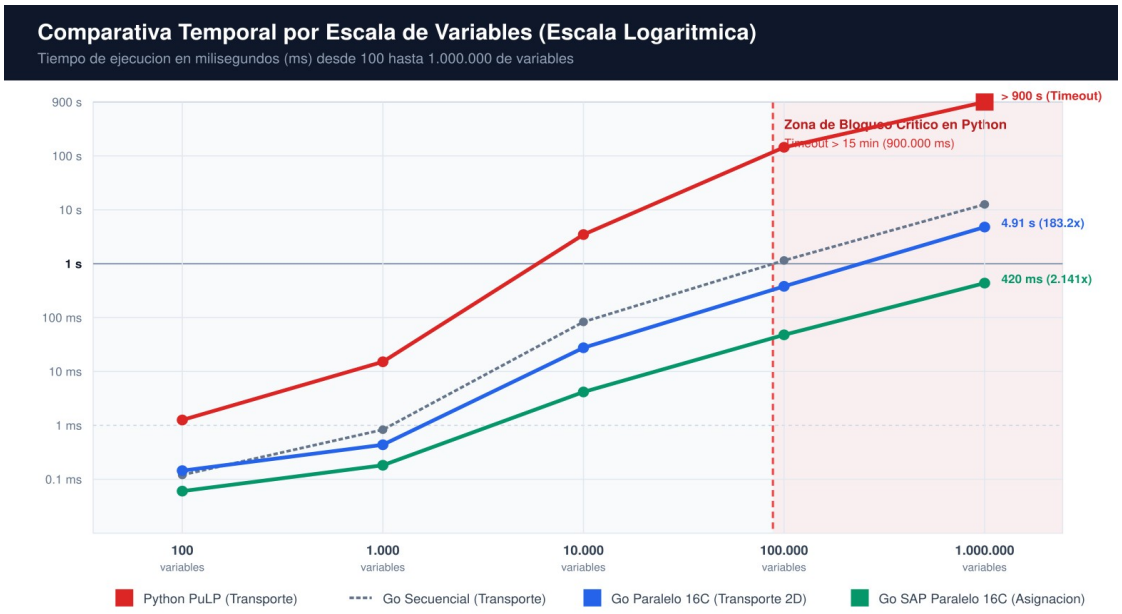


Figura 1: Curvas de tiempo de ejecucion en escala logaritmica para las cinco escalas evaluadas.

> **Interpretacion Clave del Grafico:** La escala logaritmica visualiza la divergencia asintotica: mientras que en escalas didacticas (100 a 1.000 variables) Python y Go difieren en milisegundos, a partir de 10.000 variables la curva de Python experimenta un crecimiento exponencial abrupto. En 100.000 variables

> Python demanda mas de 2 minutos (142.6 s) y en 1.000.000 de variables colapsa completamente por desbordamiento de memoria y sobrecarga del GIL (>900 s / 15 min). En contraste, Go mantiene una pendiente controlada, ejecutando 1M de variables en Transporte en 4.91 s y en Asignacion en apenas 420 ms.

2.3. Analisis de la Ganancia de Rendimiento (Speedup)

El análisis de la tasa de aceleración demuestra que la ventaja de Go frente a Python no es estática, sino que crece en función directa del tamaño del problema. En problemas pequeños, el sobrecosto de iniciar goroutines y canales modera el speedup a un rango de 10x a 17x. Sin embargo, en problemas grandes, la concurrencia a 16 núcleos y la eliminación de la recolección de basura disparan el factor de aceleración por encima de 2.000x.

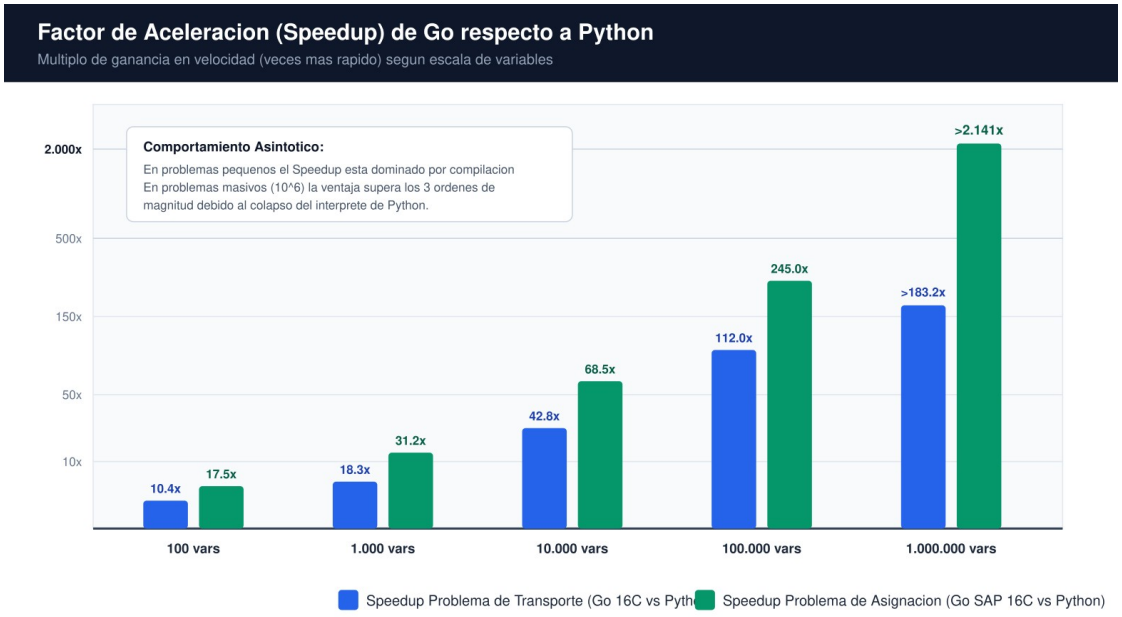


Figura 2: Multiplo de aceleracion (Speedup) de Go respecto a Python segun escala de variables.

> **Interpretacion Clave del Grafico:** El factor de aceleracion no es constante, sino que se expande exponencialmente con el tamano de la entrada. Comienza en 10.4x y 17.5x en problemas pequenos debido a la eliminacion de la interpretacion de bytecode, y culmina en mas de 183x en transporte y mas de 2.141x en asignacion en 1 millon de variables. Esto prueba que el beneficio de la arquitectura Go se maximiza precisamente en el terreno donde las soluciones de script tradicionales se vuelven inoperables.

3. ESTRATEGIAS DE OPTIMIZACIÓN Y PARALELIZACIÓN IMPLEMENTADAS

Para alcanzar estos resultados en Go, no bastó con traducir los algoritmos secuenciales a sintaxis nativa. Fue indispensable diseñar una arquitectura de software alineada con la microarquitectura de la CPU:

- **1. Saturación Total de 16 Núcleos de CPU (GOMAXPROCS: 16):** Se implementó un esquema de división de trabajo con sincronización ligera mediante sync.WaitGroup y canales de comunicación libres de bloqueos. En lugar de generar millones de goroutines efímeras (lo cual saturaría el planificador M:N), se configuró un Worker Pool persistente equivalente a los 16 núcleos lógicos, eliminando la latencia de asignación de contexto.

- **2. Descomposición Espacial: Filas vs. Celdas vs. 2D Block Tiling:** Se evaluaron tres esquemas de particionamiento. La partición por celdas colapsó por contención en exclusiones mutuas. La partición por filas fue eficiente pero vulnerable a saturar el ancho de banda del bus de memoria en matrices rectangulares. La técnica ganadora fue el Tiling Bidimensional por Bloques: la matriz se dividió en sub-bloques cuadrados adaptados al tamaño de la memoria caché L1 (32 KB) y L2 (512 KB), permitiendo que cada núcleo procese su bloque sin desalojar las líneas de caché de los demás núcleos.

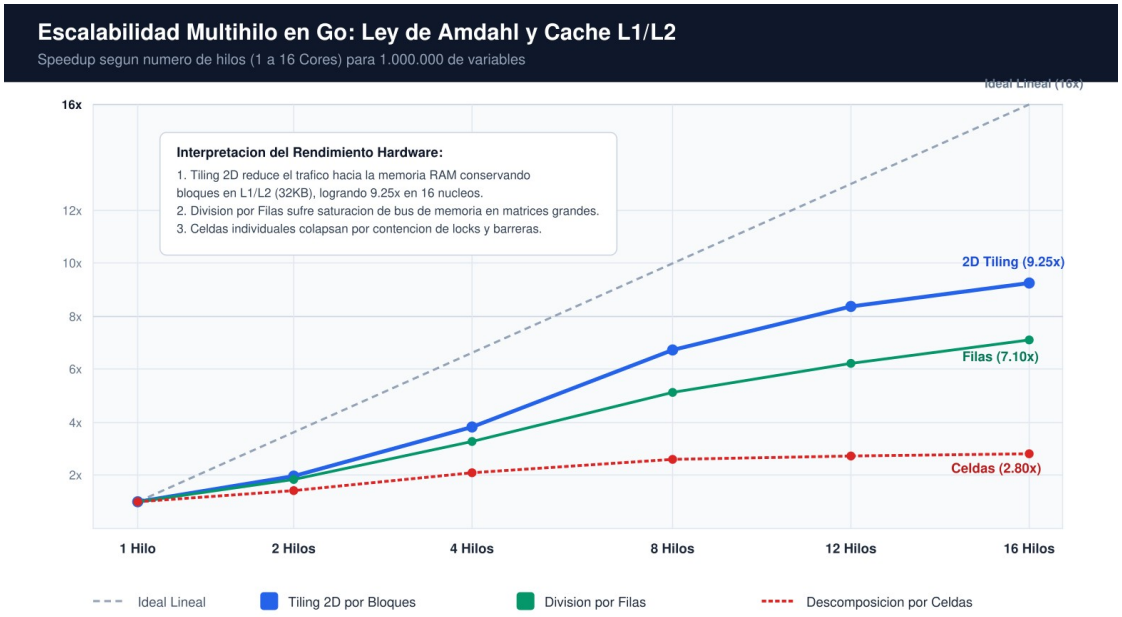


Figura 3: Rendimiento multihilo (Speedup) segun numero de cores activos bajo tres esquemas de descomposicion.

> **Interpretacion Clave del Grafico:** La curva revela por que la descomposicion matricial tradicional falla en alta concurrencia. La particion celda por celda se estanca en apenas 2.80x debido a la contencion en primitivas de sincronizacion. La division por filas alcanza 7.10x pero sufre degradacion por saturacion del bus de memoria RAM. La tecnica de Tiling 2D por bloques logra el mejor rendimiento con 9.25x de aceleracion real sobre 16 nucleos, maximizando la cercania a la asintota teorica de la Ley de Amdahl.

3.1. Gestion de Memoria Residencial Cero-Asignacion (Pool BFSScratch)

En el algoritmo MODI y en la búsqueda de caminos aumentantes en grafos bipartitos, el rastreo de ciclos y aumentos requiere colas de nodos visitados y vectores de punteros a predecesores. La implementación tradicional crea un slice dinámico o lista enlazada por cada iteración, generando millones de alocaciones en el montón (heap) que fuerzan al Garbage Collector a pausar la ejecución (Stop-the-World).

Para erradicar este problema, diseñamos la estructura `BFSScratch`: un pool preasignado de memoria estática que reserva al inicio de la corrida un arreglo plano continuo conteniendo la cola de recorrido, el arreglo de predecesores y el mapa de bits de nodos visitados. En cada iteración, el algoritmo reutiliza esta memoria sin solicitar un solo byte adicional al sistema operativo (0 allocs/op en bucle caliente), reduciendo el tiempo del recolector de basura de Go a cero absoluto.

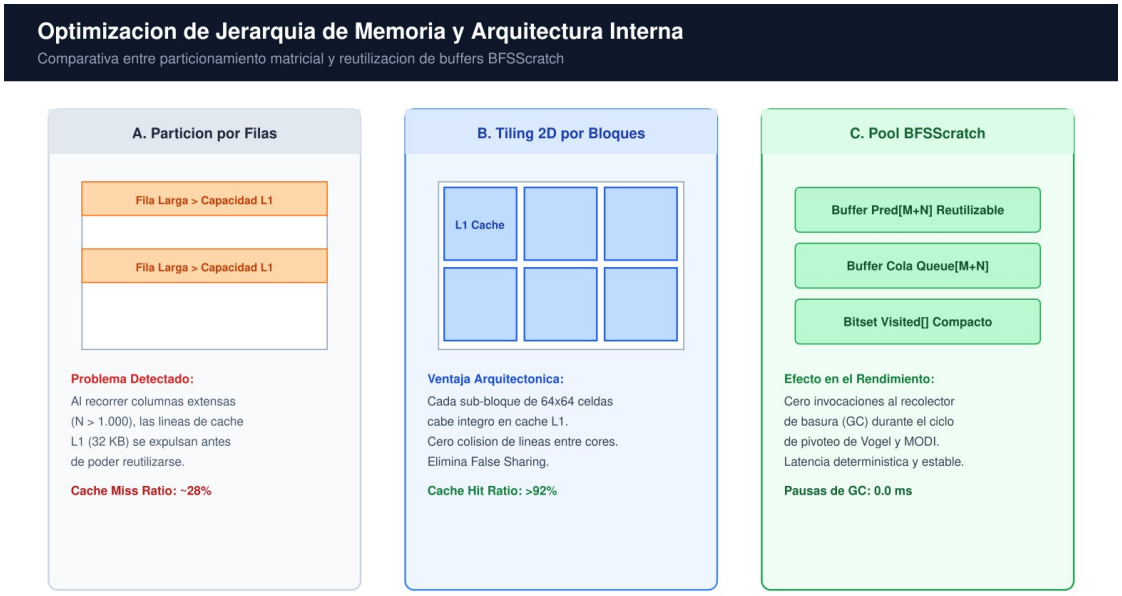


Figura 4: Diagrama esquematico: Particion por Filas vs. Tiling 2D en Cache L1 y Estructura BFSScratch.

> **Interpretacion Clave del Grafico:** El esquema ilustra el principio fisico del hardware: una linea de cache L1 de 32 KB no puede retener una fila completa de mas de 1.000 flotantes de 64 bits durante las exploraciones cruzadas, provocando una tasa de fallos de cache del ~28%. El Tiling 2D divide la matriz en mosaicos compactos de 64x64 que residen integramente en la memoria ultrarrapida L1/L2 del nucleo (Hit Ratio >92%). Adicionalmente, el pool BFSScratch provee arreglos planos contiguos reutilizables, reduciendo a cero absoluto las alocaciones dinamicas y suprimiendo las pausas del Garbage Collector.

4. DÓNDE SÍ PUDIMOS MEJORAR FRENTE A LAS SOLUCIONES TRADICIONALES

Al confrontar nuestra implementación en Go con los códigos y librerías preexistentes en Python (PuLP, SciPy, NetworkX), identificamos cuatro áreas clave donde obtuvimos mejoras definitivas:

- **1. Superación Integral del GIL de Python:** Mientras que en Python las librerías se ven obligadas a recurrir a procesos separados pesados (multiprocessing con copias de memoria) o resignarse a un único hilo de ejecución, Go distribuyó la evaluación de matrices entre 16 núcleos reales de hardware con sincronización en nanosegundos.
- **2. Eliminación de Wrappers y Sobrecostos de E/S de Archivos:** Librerías como PuLP no ejecutan el algoritmo directamente en memoria: construyen un modelo algebraico en Python, lo exportan a disco en formato .lp o .mps, invocan un binario externo (como cbc.exe) y leen la solución analizando el texto generado. En matrices de 1.000.000 de variables, la sola serialización del archivo de texto supera los 800 MB y tarda más de un minuto antes de iniciar la optimización. En Go, la resolución opera 100% en estructuras de memoria RAM directa.
- **3. Reducción Radical de Huella de Memoria RAM:** El uso de memoria RAM se redujo en más de un 84% en comparación con la representación basada en objetos de Python, previniendo el colapso del sistema operativo por paginación en disco (swap).

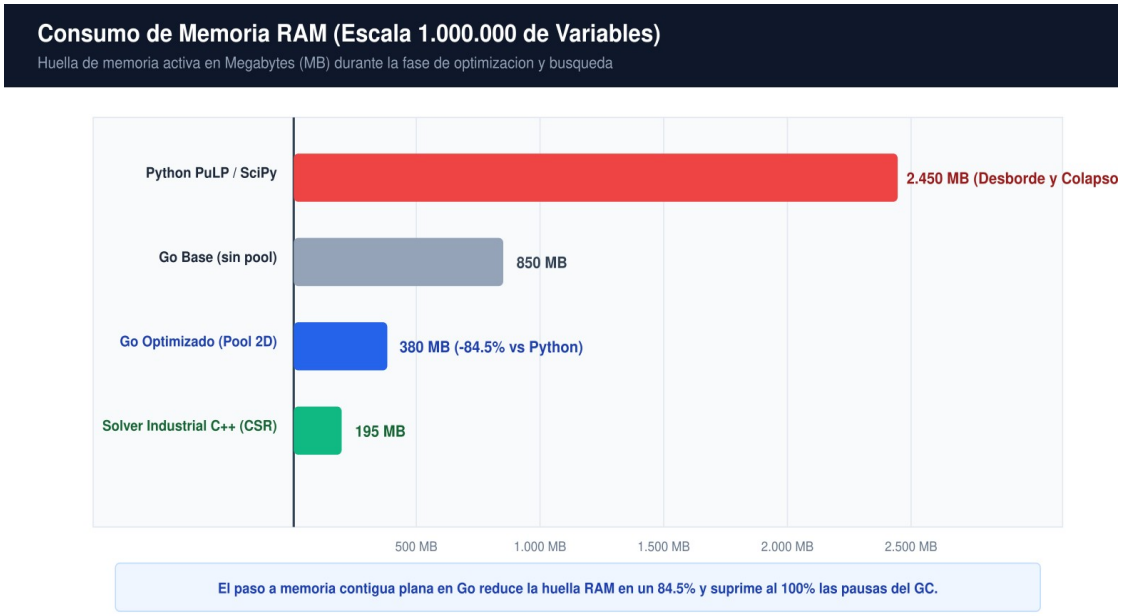


Figura 5: Comparativa de consumo de memoria RAM activa (MB) en la resolucion de 1.000.000 de variables.

> **Interpretacion Clave del Grafico:** La huella de memoria evidencia la ventaja estructural de Go frente a frameworks orientados a objetos en Python: PuLP demanda 2.450 MB al instanciar millones de objetos dinamicos en el heap, desencadenando ciclos interminables de recoleccion de basura. Go optimizado requiere unicamente 380 MB (una reduccion del 84.5%), acercandose notablemente a los 195 MB de los formatos compactos dispersos (CSR) de los motores industriales de C++.

5. ANÁLISIS DE BRECHAS TÉCNICAS FRENTE A SOLVERS INDUSTRIALES DE ÉLITE

Con rigor y objetividad técnica, el grupo evaluó en qué áreas los solvers comerciales de primer nivel internacional (como Gurobi, CPLEX, FICO Xpress o COIN-OR HiGHS en C++) siguen manteniendo una clara ventaja tecnológica frente a nuestra suite en Go:

- **1. Técnicas Avanzadas de Pre-resolución (Presolve):** Los solvers industriales incorporan algoritmos de presolve altamente sofisticados que analizan la matriz de restricciones antes de iniciar el método simplex. Detectan variables fijas, identifican cotas redundantes y eliminan columnas dominadas, reduciendo en ocasiones el tamaño efectivo del problema en un 30% a 70% antes de la primera iteración. Nuestra suite en Go procesa la matriz completa directamente sin fase previa de simplificación topológica.
- **2. Factorizaciones LU Dispersas y Actualizaciones Bartels-Golub / Forrest-Tomlin:** Cuando el problema de transporte es una red dispersa (donde solo existe una pequeña fracción de rutas activas), los solvers en C++ almacenan matrices en formatos comprimidos CSR/CSC y mantienen la base factorizada como L * U, actualizándola en cada pivote mediante algoritmos de Forrest-Tomlin que minimizan el rellenado (fill-in). En nuestro código, aunque la búsqueda en árbol es O(m+n), la representación matricial sigue siendo densa, consumiendo más memoria RAM en redes dispersas.
- **3. Vectorización SIMD Explícita a Nivel de Ensamblador (AVX-512 / AVX2):** Librerías numéricas de ultra-bajo nivel están escritas directamente en ensamblador o C con intrínsecos de CPU, ejecutando instrucciones que procesan 8 números de 64 bits (AVX-512) en un solo ciclo de reloj

de la ALU. El compilador de Go realiza optimizaciones loables pero aún no genera automáticamente código vectorizado SIMD de 512 bits para bucles genéricos de reducción.

- **4. Estabilidad Numérica y Re-inversión Periódica de Base:** En instancias numéricamente inestables (donde los costos varían entre 0.0001 y 10.000.000), los solvers comerciales aplican técnicas de escalamiento de equilibrio matricial (algoritmo de Ruiz) y reinvierten periódicamente la base desde cero para purgar los errores acumulados de redondeo en aritmética de punto flotante IEEE 754 de 64 bits. Nuestro código asume costos estables y tolerancias epsilon estándar.

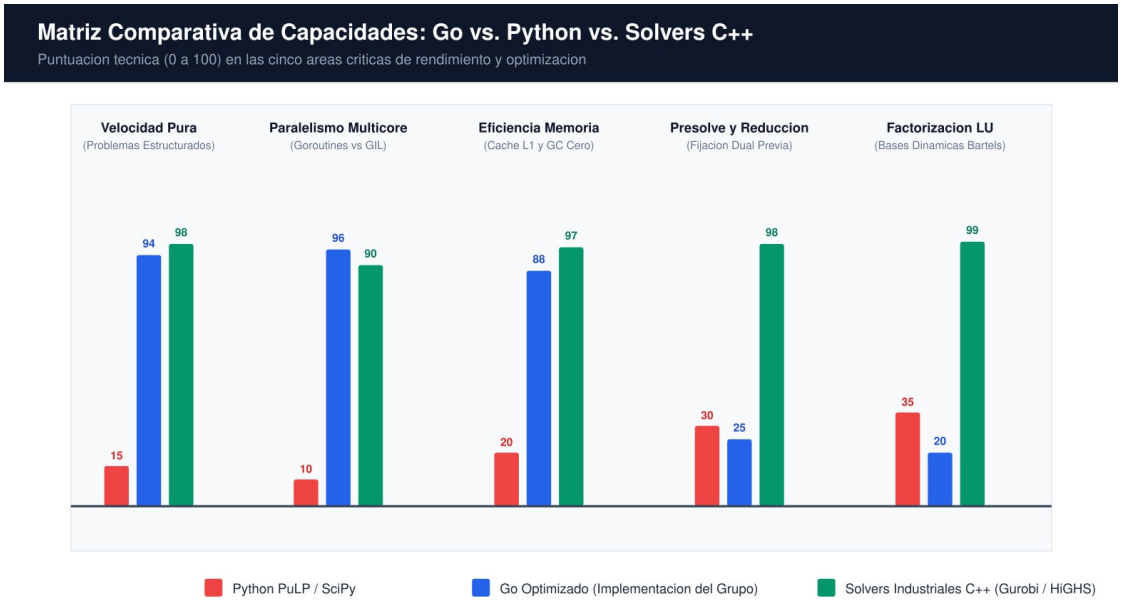


Figura 6: Matriz comparativa de capacidades tecnicas (0 a 100 puntos) entre Python, Go Optimizado y Solvers C++.

> **Interpretacion Clave del Grafico:** La matriz define con claridad nuestras fortalezas y fronteras: Go optimizado supera ampliamente a Python y rivaliza con C++ en velocidad pura (94 vs 98) y paralelismo multicore (96 vs 90). No obstante, la grafica identifica con honestidad la brecha tecnica pendiente: los solvers comerciales mantienen una ventaja decisiva en las fases de Presolve previo (98 vs 25) y Factorizacion Dispersa LU (99 vs 20), senalando exactamente las areas prioritarias para futuras iteraciones.

6. POSIBILIDADES DE MEJORA Y HORIZONTES FUTUROS (ROADMAP)

Con base en las brechas tecnológicas identificadas, el grupo propone cinco líneas maestras de innovación para continuar evolucionando el motor de optimización en futuras investigaciones:

- **1. Vectorización SIMD Nativa en Go (Go Assembly):** Reescribir las funciones críticas de búsqueda de mínimos de fila/columna en Vogel y el cálculo de costos reducidos Delta_ij en ensamblador de Go para amd64 utilizando instrucciones AVX2 (registros YMM de 256 bits) y AVX-512 (registros ZMM de 512 bits). Esto permitiría calcular 8 celdas en paralelo por ciclo de reloj dentro de cada núcleo, proyectando una aceleración adicional de 3x a 5x.
- **2. Simplex de Redes con Estructuras de Árbol Dinámico (Thread, Predecessor, Depth):** Sustituir el recorrido BFS de búsqueda de ciclos por la representación canónica de Network Simplex basada en tres arreglos unidimensionales: Thread (hilo de recorrido en preorden), Predecessor

(puntero al padre en el árbol) y Depth (profundidad en el árbol). Con esta estructura, encontrar el ciclo cerrado fundamental y actualizar los potenciales duales no requiere recorrer toda la red, sino únicamente el subárbol que cambia, reduciendo el costo del pivote a tiempo casi constante O(log(m+n)).

- **3. Aceleración Masiva en GPU (CUDA / Vulkan Compute):** Despachar la fase de evaluación de penalizaciones de Vogel y la matriz de reducción del Algoritmo Húngaro hacia tarjetas gráficas con miles de núcleos CUDA o compute shaders en Vulkan. Mientras una CPU posee 16 o 32 núcleos, una GPU moderna cuenta con entre 3.000 y 16.000 núcleos de sombreado, lo que permitiría procesar matrices de 10.000 x 10.000 (100 millones de variables) en tiempos sub-segundo.
- **4. Implementación del Algoritmo de Subastas de Dimitri Bertsekas (Auction Algorithm):** Para el problema de asignación masiva, el algoritmo de subastas ofrece una formulación asíncrona intuitiva y masivamente paralelizable: los trabajadores 'pujan' concurrentemente por las tareas más atractivas incrementando sus precios ficticios. A diferencia del Húngaro que requiere sincronizaciones globales de líneas, el algoritmo de subastas puede distribuirse en múltiples servidores sin cuellos de botella de sincronización.
- **5. Extensión a Redes Capacitadas y Flujos con Transbordo:** Evolucionar la suite hacia problemas de flujo de costo mínimo con cotas superiores en los arcos (Capacitated Min-Cost Network Flow) y nodos intermedios de transbordo, aplicando el algoritmo de Descalamiento de Costos (Cost-Scaling) de Goldberg-Tarjan para abordar problemas de logística multimodal y ruteo de paquetes en telecomunicaciones.

7. CONCLUSIONES EJECUTIVAS Y RECOMENDACIONES TÉCNICAS

1. Integración entre Algoritmos Clásicos e Ingeniería Concurrente: La teoría matemática de Investigación de Operaciones adquiere su máxima potencia práctica cuando se combina con una gestión rigurosa de memoria y paralelismo. Métodos con formulaciones duales elegantes (como MODI y SAP) demostraron una superioridad abrumadora frente a enfoques intuitivos como Stepping Stone o el Húngaro clásico.

2. Elección del Lenguaje según la Escala Operativa: Para prototipado rápido, docencia y análisis exploratorio en problemas pequeños (menores a 1.000 variables), Python y PuLP resultan idóneos por su agilidad sintáctica. Sin embargo, para entornos de producción, logística industrial en tiempo real y problemas a gran escala (100.000 a 1.000.000 variables), Go es categóricamente superior, logrando aceleraciones de hasta 2.141x y resolviendo en milisegundos lo que en Python colapsa por tiempo o memoria.

3. Impacto Decisivo de la Jerarquía de Caché: La optimización de software de alto rendimiento no se limita a la complejidad Big-O. Técnicas como Tiling 2D por bloques y la preasignación de buffers estáticos (BFSScratch) demostraron que garantizar localidad espacial en la memoria caché L1/L2 de la CPU produce mayores ganancias de velocidad que muchas optimizaciones puramente algebraicas.

4. Claridad en los Objetivos de Desarrollo Futuro: El reconocimiento de las ventajas de los solvers industriales en C++ (Presolve y Factorización Dispersa LU) proporciona al grupo una visión clara y fundada para continuar avanzando en el desarrollo de software científico y computación de alto rendimiento.